

Python Fundamentals for AI & ML

Most Popular Programming Language for AI Development

Units

- 1 Introduction To Python
- 2 Python Environment Setup
- 3 Python Syntax
- 4 Pandas Essentials
- 5 NumPy Essentials
- 6 Matplotlib Essentials
- 7 Examples : Data Cleaning, Preprocessing and Transformation

OVERVIEW : MODULE

Python Fundamentals for AI & ML

Learning Outcomes

- Understand Python syntax, variables, and control structures.
- Work with essential data structures like lists, tuples, and dictionaries.
- Utilize NumPy for array manipulation and Pandas for data manipulation with Data Frames.
- Create meaningful visualizations using Matplotlib.
- Perform data cleaning, preprocessing, and exploratory data analysis.



3.1

Introduction to Python

Lesson Learning Outcomes

Outcome

- ✓ Understand the definition of a programming language and it's necessity in AI & ML development.
- ✓ Know what is Python and it's history
- ✓ Understand why Python is important for AI & ML
- ✓ Know the names of important Python libraries

LESSON OVERVIEW

Python Fundamentals for AI & ML

Lesson 1 Introduction to Python

In this lesson, we will explore the fundamental role of programming languages in instructing computers and building intelligent systems like those in **AI and ML**. Specifically, we'll focus on **Python**, a widely acclaimed language renowned for its readability and versatility.

We'll delve into why Python is a preferred choice for **AI and ML**, examining its popularity, extensive library ecosystem, and flexibility for tackling diverse problems effectively.

- Programming Language
- Python and it's history
- Reason for using Python for AI and ML development
- Python libraries in AI & ML

What Is a Programming Language?

Way to Instruct Machines

A **programming language** is a system of notation for writing computer programs. It allows programmers to express instructions that computers can understand and execute.

Syntax

The form or structure of the language (how code is written).

Semantics

The meaning of the language constructs (what the code does).

Why do we need a programming language for AI & ML

Role of Programming Language

Give instructions to computers

Just like we tell our friends what to do, we use programming languages to tell computers what to do for AI and ML.

Build intelligent machines

Programming languages are like building blocks that help us create smart machines that can learn and improve. Programming languages provide tools and libraries for efficiently handling and processing data, including cleaning, transforming, and analyzing datasets.

Introduction to Python

- **High-level:** Abstracts low-level details, making it easier to write code.
- **General-purpose:** Used for various tasks (web development, data science, etc.).
- **Readability:** Emphasizes code readability with significant indentation.

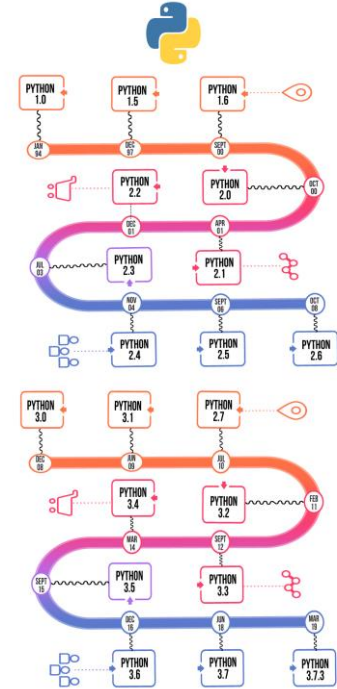
Why Python

- **Popularity:** Widely used and loved by developers.
- **Machine Learning:** Gained prominence in ML and data science.
- **Community:** Active community and extensive resources.



Brief History of Python

- **Guido van Rossum** started Python in the late 1980s.
- Released as **Python 0.9.0** in 1991.
- **Python 2.0** (2000) and **Python 3.0** (2008) brought major changes.



Why Python for AI & ML

Most Popular Language for AI & ML Development

- Python is a **popular choice** for AI and ML due to its simplicity, versatility, and powerful libraries.
- Python boasts a **vast ecosystem** of libraries specifically designed for AI and ML.
- Libraries like NumPy, Pandas, and Scikit-learn provide **comprehensive tools** for data manipulation, analysis, and model building.
- **Flexibility** allows developers to implement AI and ML algorithms in a manner that best suits their problem domain and preferences.

Introduction To Python

SUMMARY

- ✓ Definition and importance of programming languages in AI & ML
- ✓ Introduction to Python and its historical background
- ✓ Significance of Python for AI & ML applications
- ✓ Key Python libraries essential for AI & ML tasks



3.2

Python Environment Setup

Lesson Learning Outcomes

- ✓ Understand the concept of Anaconda and its importance in data science
- ✓ Understand the concept of an Integrated Development Environment (IDE)
- ✓ Explore the features and benefits of using an IDE for programming
- ✓ Understand the concept of Jupyter Notebook and its role in data science
- ✓ Learn how to create, edit, and execute code cells in a Jupyter Notebook
- ✓ Explore the features and capabilities of Jupyter Notebook for interactive computing

LESSON OVERVIEW

Python Fundamentals for AI & ML

Lesson 2 Python Environment Setup

In this lesson, we'll explore the process of setting up **Anaconda** as an environment for AI & ML development with **Python**. **Anaconda** offers a comprehensive platform that streamlines the installation of Python and essential libraries for data science tasks. By installing Anaconda, users gain access to a wide array of pre-installed packages and tools tailored specifically for AI & ML projects.

We'll delve into the steps involved in installing Anaconda and configuring it to efficiently support AI & ML workflows, providing a solid foundation for diving into Python-based data science endeavors.

- Anaconda installation
- Definition of an IDE
- Features and benefits of using an IDE.
- Jupyter Notebook

Setting Up Python Environment

Installing Python

To make our machine ready for programming, especially for AI & ML, we need to install a **package called 'Anaconda'**

Anaconda is a distribution

It's a collection of software packages specifically designed for data science, scientific computing, and machine learning. One of the key components within Anaconda is Python itself.

Convenience

By including Python, Anaconda saves you the extra step of installing Python separately. Everything you need to get started with data science using Python is bundled together.

Installing Anaconda



- Follow the steps in this link
<https://docs.anaconda.com/free/anaconda/install/windows/>

What is an IDE

A Software for Writing Code

An **Integrated Development Environment (IDE)** is software that combines commonly used developer tools into a compact GUI application.

It helps boost developer productivity by providing features like:

- **Editor:** Syntax highlighting, auto-completion, and bug checking
- **Compiler:** Translates human-readable code into machine-specific code
- **Debugger:** Assists in testing and locating errors
- **Built-in Terminal:** Allows direct interaction with the operating system
- **Version Control:** Helps manage software changes
- **Code snippets:** Reusable code segments
- **Extensions and Plugins:** Enhances functionality
- **Code navigation:** Tools for analyzing code

What Is Jupyter Notebook?

Popular IDE for Data Scientists

Jupyter Notebook is a web-based interactive development environment for creating and sharing computational documents.

Key features:

Live Code: Combine code, equations, and narrative text.

Visualizations: Interactive charts, images, and LaTeX.

Live code: Combine code, equations, and narrative text.

Multiple languages: Supports Python, R, Julia, Scala, and more.

Shareable: Email, Dropbox, GitHub, or Jupyter Notebook Viewer.

Big data integration: Leverage tools like Apache Spark.

Python Environment Setup

SUMMARY

- ✓ Anaconda is a popular open-source distribution of Python and R programming languages for data science and machine learning.
- ✓ It comes pre-packaged with a collection of essential libraries and tools, making it easy to get started with data analysis and modeling.
- ✓ We learned how to install Anaconda on different operating systems and explored Anaconda Navigator, a graphical user interface for managing environments and packages.
- ✓ IDEs are software applications that provide comprehensive facilities for software development.
- ✓ We discussed the concept of IDEs and their benefits, such as code editing, debugging, and project management.
- ✓ Jupyter Notebook is a web-based interactive computing environment that enables users to create and share documents containing live code, equations, visualizations, and narrative text.
- ✓ We explored the concept of Jupyter Notebook and its advantages for interactive data analysis and exploration.
- ✓ Basic operations in Jupyter Notebook, such as creating, editing, and executing code cells, were covered. Additionally, we discussed Markdown cells for text formatting and visualization capabilities using libraries like matplotlib.



3.3

Python Syntax

Lesson Learning Outcomes

- ✓ Understand Python syntax and variables, including basic data types
- ✓ Perform basic arithmetic operations and use control structures for decision making and iteration
- ✓ Utilize essential data structures like lists, tuples, and dictionaries for efficient data storage and retrieval
- ✓ Define functions and apply them to modularize your code and enhance reusability
- ✓ Grasp the basics of object-oriented programming (OOP) and its application in Python

LESSON OVERVIEW

Python Fundamentals for AI & ML

Lesson 3 Python Syntax

In this lesson, we'll explore the crucial steps of data collection and preprocessing in the realm of **Artificial Intelligence (AI)** and **Machine Learning (ML)**. We'll start by defining data collection and preprocessing techniques, emphasizing their importance in ensuring the quality and reliability of data analysis and ML models. Next, we'll discuss the challenges associated with data, including inaccuracies, missing values, imbalances, and biases.

Finally, we'll delve into practical considerations and methods for effective data collection, highlighting various approaches and common issues encountered.

- Python variables, data types, operations
- Python loops, functions
- Object Oriented Programming

Variables and Data Types

Defining 'Things'

Variables store data values and have a name.

Data types include :

- **Integers**
- **floats**
- **Strings**
- **Booleans**
- **Lists**
- **Tuples**
- **Dictionaries**

```
# Variable assignment
x = 10
name = "Alice"

# Data types
num = 3.14
is_true = True
my_list = [1, 2, 3]
my_dict = {"name": "Bob", "age": 25}
```

Basic Operations

Different Types of Operators

Arithmetic operators

+, -, *, /, %, **

Comparison operators

==, !=, <, >, <=, >=

Logical operators

and, or, not

```
# Basic operations
result = 10 + 5
is_equal = (3 + 2) == (2 * 3)
is_greater = 10 > 5 and 3 < 2
```

Control Flow - Conditional Statements

Programming Conditions

- Use **if, elif, else** for decision making
- **Indentation** is crucial in Python

```
# Conditional statements
age = 20

if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

Control Flow - Loops

Making The Machine Work In Loops

- Use **for** and **while** loops for iteration
- **Break** and **continue** statements modify loop behavior

```
# For loop
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# While loop
count = 0
while count < 5:
    print(count)
    count += 1
```

Functions

Piece of Re-usable Instruction

- Functions are **reusable** blocks of code
- Define functions using **def** keyword

```
# Function definition
def greet(name):
    print("Hello, " + name + "!")

# Function call
greet("Alice")
```


Lists and Tuples

Data Containers

- **Lists** are mutable sequences, and tuples are immutable sequences
- **Access elements** using indexing and slicing

```
# Lists and Tuples
my_list = [1, 2, 3]
my_tuple = (4, 5, 6)

# Indexing and Slicing
first_element = my_list[0]
sublist = my_list[1:3]
```

Dictionaries

Data Containers

- Dictionaries store **key-value pairs**
- Keys are **unique** and **immutable**

```
# Dictionaries
my_dict = {"name": "Alice", "age": 30}

# Accessing values
print(my_dict["name"])
```

String Manipulation

Working with Texts

- Strings are immutable **sequences of characters**
- Use string methods for **manipulation**

```
# String manipulation
my_string = "Hello, World!"

# String methods
upper_case = my_string.upper()
```

List Comprehensions

List Generators

- **Concise way** to create lists
- Syntax: [expression for item in **iterable** if condition]

```
# List comprehension  
squares = [x ** 2 for x in range(1, 6)]
```

Error Handling

Mistake Management Mechanism

- Use try-except blocks to **handle errors** gracefully
- **Prevents programs** from crashing due to unexpected errors

```
# Error handling
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

Modules and Packages

Toolboxes and Kits

- Modules are Python files containing **functions and variables**
- Packages are **directories** of Python modules

```
# Importing modules
import math

# Using functions from modules
print(math.sqrt(16))
```

Object-Oriented Programming (OOP)

Building with Blocks

- OOP is a programming **paradigm** based on objects and classes

```
# Class definition
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print("Hello, my name is", self.name)

# Object creation
person1 = Person("Alice", 30)
person1.greet()
```

Python Syntax

SUMMARY

- ✓ Covered Python fundamentals including variables, data types, control structures, and functions.
- ✓ Introduced object-oriented programming principles.



3.4

Pandas Essentials

Lesson Learning Outcomes

- ✓ Understand the importance of Pandas in data manipulation and analysis
- ✓ Learn how to create, manipulate, and analyze data using Pandas
- ✓ Gain familiarity with essential Pandas functions and techniques for data cleaning, preprocessing, and exploration
- ✓ Develop the skills to handle real-world datasets effectively for data science tasks

LESSON OVERVIEW

Python Fundamentals for AI & ML

Lesson 4 Pandas Essentials

This lesson introduces **Pandas**, a Python library for **data analysis**. We explore its core data structures, Series and DataFrame, and learn how to manipulate data effectively. Topics include handling missing data, data aggregation, and grouping, as well as data visualization techniques.

Through practical examples and case studies, we gain hands-on experience applying Pandas to real-world scenarios.

- Introduction to Pandas
- Data Structures in Pandas: Series and DataFrame
- Data Manipulation with Pandas
- Handling Missing Data
- Data Aggregation and Grouping

Introduction to Pandas



Pandas is a Python library for data manipulation and analysis. It provides powerful data structures and tools for working with structured data.

Key Pandas Data Structures

- **Series:** 1-dimensional labeled array capable of holding any data type.
- **DataFrame:** 2-dimensional labeled data structure with columns of potentially different types.

Installing Pandas

- Pandas can be installed using **pip**, the Python package installer
- On Windows: **Press Win + R**, type "**cmd**", and **press Enter**
- Type the following command and **press Enter**: pip install pandas
- **Verify Installation**

```
import pandas as pd
print(pd.__version__)
```

Creating a series

```
import pandas as pd

data = [1, 2, 3, 4, 5]
series = pd.Series(data)
print(series)
```

Creating a DataFrame

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie', 'David'],
        'Age': [25, 30, 35, 40],
        'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']}

df = pd.DataFrame(data)
print(df)
```

Loading Data into a DataFrame

```
import pandas as pd

# Load CSV file
df = pd.read_csv('data.csv')

# Load Excel file
df = pd.read_excel('data.xlsx')

# Load JSON file
df = pd.read_json('data.json')
```

Basic DataFrame Operations

```
import pandas as pd

# View first few rows
print(df.head())

# View last few rows
print(df.tail())

# Get information about the DataFrame
print(df.info())

# Generate descriptive statistics
print(df.describe())
```


Indexing and Slicing

- Use **loc[]** for label-based indexing
- Use **iloc[]** for positional indexing

```
# Select a single column by label
print(df['Name'])

# Select a single column by position
print(df.iloc[:, 0])

# Select rows and columns by label
print(df.loc[1:3, ['Name', 'Age']])

# Select rows and columns by position
print(df.iloc[1:3, 0:2])
```

Missing Data Handling

Pandas provides methods to handle missing data, denoted as NaN.

- **dropna():** Drop rows with missing values
- **fillna():** Fill missing values with a specified value

```
# Drop rows with missing values
df.dropna()

# Fill missing values with a specified value
df.fillna(0)
```

Pandas Essentials

SUMMARY

- ✓ Pandas is a powerful Python library for data manipulation and analysis.
- ✓ It provides data structures like Series and DataFrame, which are versatile and efficient for handling structured data.
- ✓ With Pandas, you can perform various data operations, including data cleaning, preprocessing, transformation, and analysis.
- ✓ Pandas offers functionalities for handling missing data, grouping, aggregating, and visualizing data.
- ✓ Through practical examples and case studies, you've gained valuable skills in working with real-world datasets effectively.



3.5

Numpy Essentials

Lesson Learning Outcomes

- ✓ Understand the importance of NumPy in numerical computing and data manipulation
- ✓ Learn how to efficiently work with arrays and matrices using NumPy
- ✓ Gain proficiency in performing mathematical and statistical operations on numerical data
- ✓ Develop skills in array manipulation, slicing, indexing, and broadcasting
- ✓ Explore advanced topics in NumPy, such as linear algebra, random number generation

LESSON OVERVIEW

Python Fundamentals for AI & ML

Lesson 5 Numpy Essentials

This module introduces **array creation** and **manipulation** in **Python**, along with mathematical operations and functions.

- Array Creation and Manipulation
- Mathematical Operations and Functions
- Data Aggregation and Statistics

Introduction to Numpy

- NumPy is a **powerful Python library** for numerical computing.
- It provides **support** for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.
- NumPy's arrays are significantly **faster and more memory-efficient** than Python lists.
- It is the **foundation** of many other Python libraries used in data analysis, machine learning, and scientific computing.



Computing Element-wise Operations

```
import numpy as np

# Create two arrays
a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

# Element-wise addition
result = a + b
print(result)
```

Broadcasting

NumPy **automatically** broadcasts arrays of different shapes during arithmetic operations, making it easy to work with data of different dimensions.

```
import numpy as np

# Create an array and scalar
a = np.array([1, 2, 3, 4])
scalar = 10

# Scalar multiplication
result = a * scalar
print(result)
```


Key NumPy Features

Random Number Generation

- NumPy provides functions for generating random numbers from various probability distributions
- Useful for simulations, random sampling, and statistical analysis

```
import numpy as np

# Generate 5 random numbers from a uniform
distribution
random_numbers = np.random.uniform(size=5)
print(random_numbers)
```

Key NumPy Features

Linear Algebra Operations

- NumPy provides a **comprehensive suite** of linear algebra functions, including matrix multiplication, eigenvalues, and decompositions
- Essential for **data science tasks** involving linear algebra, such as regression and dimensionality reduction

```
import numpy as np

# Create two matrices
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
result = np.dot(A, B)
print(result)
```

Numpy Essentials

SUMMARY

- ✓ Acquired essential skills in numerical computing and data manipulation through NumPy.
- ✓ Developed proficiency in leveraging NumPy for various data science tasks.
- ✓ Equipped with knowledge for effective mathematical computing using NumPy.
- ✓ Ready to apply learned techniques in practical scenarios to further solidify proficiency.
- ✓ Prepared to tackle diverse data manipulation tasks with confidence using NumPy.

3.6

Matplotlib Essentials

Lesson Learning Outcomes

- ✓ Proficiency in creating a variety of plots using Matplotlib
- ✓ Practical skills in leveraging Matplotlib for data visualization tasks in data analysis, research, and presentations

LESSON OVERVIEW

Python Fundamentals for AI & ML

Lesson 6 Matplotlib Essentials

This lesson provides a comprehensive understanding of **data visualization's** significance in data analysis and interpretation. We delve into Matplotlib's architecture and its pivotal role in crafting visual representations of data.

Throughout the module, we learn to create fundamental plots, laying the groundwork for more intricate visualizations in subsequent lessons.

- Understanding the importance of data visualization in data analysis and interpretation.
- Overview of Matplotlib's architecture and its role in creating visualizations.
- Learning how to create basic plots,

Introduction to Matplotlib

- Matplotlib is a comprehensive library for creating **static, animated, and interactive visualizations** in Python.
- It provides a **wide variety of plotting functions** to visualize data in 2D and 3D formats.
- Matplotlib **enables data scientists** and machine learning practitioners to explore and communicate insights from their data effectively.
- Visualizations play a crucial role in understanding **patterns, trends, and relationships** within data.



Scatter Plot

```
import matplotlib.pyplot as plt

# Sample data
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# Create a scatter plot
plt.scatter(x, y)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Scatter Plot')
plt.show()
```

Line Plot

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Create a line plot
plt.plot(x, y, color='blue', linestyle='--', marker='o',
label='sin(x)')

plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Line Plot')
plt.legend()
plt.grid(True)
plt.show()
```


Subplots

```
import numpy as np
import matplotlib.pyplot as plt

# Generate data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# Create subplots
plt.subplot(2, 1, 1)

plt.plot(x, y1, color='blue')
plt.title('Sin(x)')

plt.subplot(2, 1, 2)
plt.plot(x, y2, color='red')
plt.title('Cos(x)')

plt.tight_layout()
plt.show()
```

Matplotlib Essentials

SUMMARY

- ✓ Introduction to Matplotlib
- ✓ Line Plot
- ✓ Scatter Plot
- ✓ Subplots



3.7

Examples : Data Cleaning, Preprocessing and Transformation

Lesson Learning Outcomes

- ✓ Proficiency in identifying and handling missing data, duplicate records, and outliers through effective data cleaning techniques
- ✓ Ability to preprocess raw data to ensure it meets the requirements for analysis and modeling, including scaling, normalization, and encoding categorical variables
- ✓ Understanding of how data transformation techniques can enhance the quality and predictive power of machine learning models
- ✓ Practical skills in implementing data cleaning, preprocessing, and transformation techniques using Python libraries such as Pandas and Scikit-learn
- ✓ Preparedness to apply these techniques to real-world datasets, ensuring data quality and improving the performance of machine learning models

LESSON OVERVIEW

Python Fundamentals for AI & ML

Lesson 7 Examples : Data Cleaning, Preprocessing and Transformation

This lesson covers examples of **data preprocessing techniques**, including handling missing data, duplicates, and outliers effectively.

We explore scaling and normalization methods for consistency across features. Additionally, we delve into data transformation, and enhancing interpretability.

- Exploring techniques for handling missing data, duplicate records, and outliers effectively.
- Exploring techniques for scaling, normalization
- Understanding the concept of data transformation and its role in enhancing the interpretability and predictive power of models.

Data Cleaning and Preprocessing Basics

Handling Missing Values

Dropna(): Remove rows or columns with missing values.

Fillna(): Fill missing values with a specific value or using interpolation.

```
import pandas as pd

# Example: Handling missing values
data = {'A': [1, 2, None, 4],
        'B': [5, None, 7, 8]}
df = pd.DataFrame(data)

# Drop rows with missing values
df.dropna()

# Fill missing values with a specific value
df.fillna(0)
```

Data Transformation

- **Scaling:** Scale features to a similar range (e.g., Min-Max scaling, Standard scaling).
- **Normalization:** Scale features to have a mean of 0 and variance of 1.

```
import numpy as np

# Example: Min-Max scaling with numpy
data = np.array([[1, 2],
                 [3, 4],
                 [5, 6]])

min_val = np.min(data, axis=0)
max_val = np.max(data, axis=0)

scaled_data = (data - min_val) / (max_val - min_val)
print(scaled_data)
```

Introduction to Exploratory Data Analysis (EDA) with Python

Exploring Data

- **Exploratory Data Analysis (EDA)** is an essential step in understanding the characteristics of the data.
- It involves **summarizing the main characteristics** of the data, often with visual methods.
- EDA helps **identify patterns, relationships, and anomalies** in the data, guiding further analysis and modeling.

Defining an example dataframe

```
import pandas as pd
import numpy as np

# Define a sample DataFrame for student data
data = {
    'Student_ID': np.arange(1, 6),
    'Name': ['John', 'Emily', 'Michael', 'Sophia', 'William'],
    'Age': [20, 21, 19, 22, 20],
    'Grade': [3.8, 3.9, 3.5, 4.0, 3.7],
    'Major': ['Computer Science', 'Biology', 'Mathematics', 'Engineering', 'Psychology']
}

student_df = pd.DataFrame(data)
```


Descriptive Statistics

- **Mean, median, mode, standard deviation, variance, etc.**
- **Provides a quick overview of central tendency, spread, and shape of the data distribution.**

```
# Example: Summary statistics with Pandas
summary_stats = student_df.describe()
print(summary_stats)
```

Data Distribution

- **Histograms, box plots, density plots, etc.**
- **Visualize the distribution of individual variables.**

```
import matplotlib.pyplot as plt

# Example: Histogram with Matplotlib
plt.hist(student_df['Age'], bins=10)
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Histogram of Age')
plt.show()
```

Data Exploration

Outlier Detection and Handling

Box Plots

- **Identify outliers** based on the distribution of data.
- Outliers may **indicate errors** in data collection or interesting phenomena.

Z-Score or IQR Method

Calculate z-scores or use interquartile range (IQR) to identify and handle outliers.

```
# Example: Box plot with Matplotlib
plt.boxplot(student_df['Grade'])
plt.title('Box Plot of Grade')
plt.show()

# Example: Detect outliers using z-score with NumPy
z_scores = (student_df['Grade'] -
np.mean(student_df['Grade'])) /
np.std(student_df['Grade'])

outliers = student_df[(z_scores > 3) | (z_scores <
-3)]
```

Examples : Data Cleaning, Preprocessing and Transformation

SUMMARY

- ✓ Covered foundational concepts in data preprocessing through simple code examples.
- ✓ Explored handling missing data, duplicates, and outliers using effective techniques.
- ✓ Introduced basic methods for scaling and normalization to ensure data consistency.
- ✓ Provided clear explanations of data transformation, enhancing understanding of its role in model interpretability and predictive power.

Key Takeaways

- Python offers a **powerful toolkit** for data science tasks, from data manipulation to visualization.
- Setting up Python environment through **Anaconda**.
- **Data types, syntax and object oriented programming** in Python.
- Understanding Python **key libraries** like NumPy, Pandas, and Matplotlib.
- Basic examples of **Data Cleaning, Preprocessing** and **Transformation**.